

Computer Organization & Assembly Language (CSC-205)

Lecture 1: Orientation & Introduction

Course Orientation

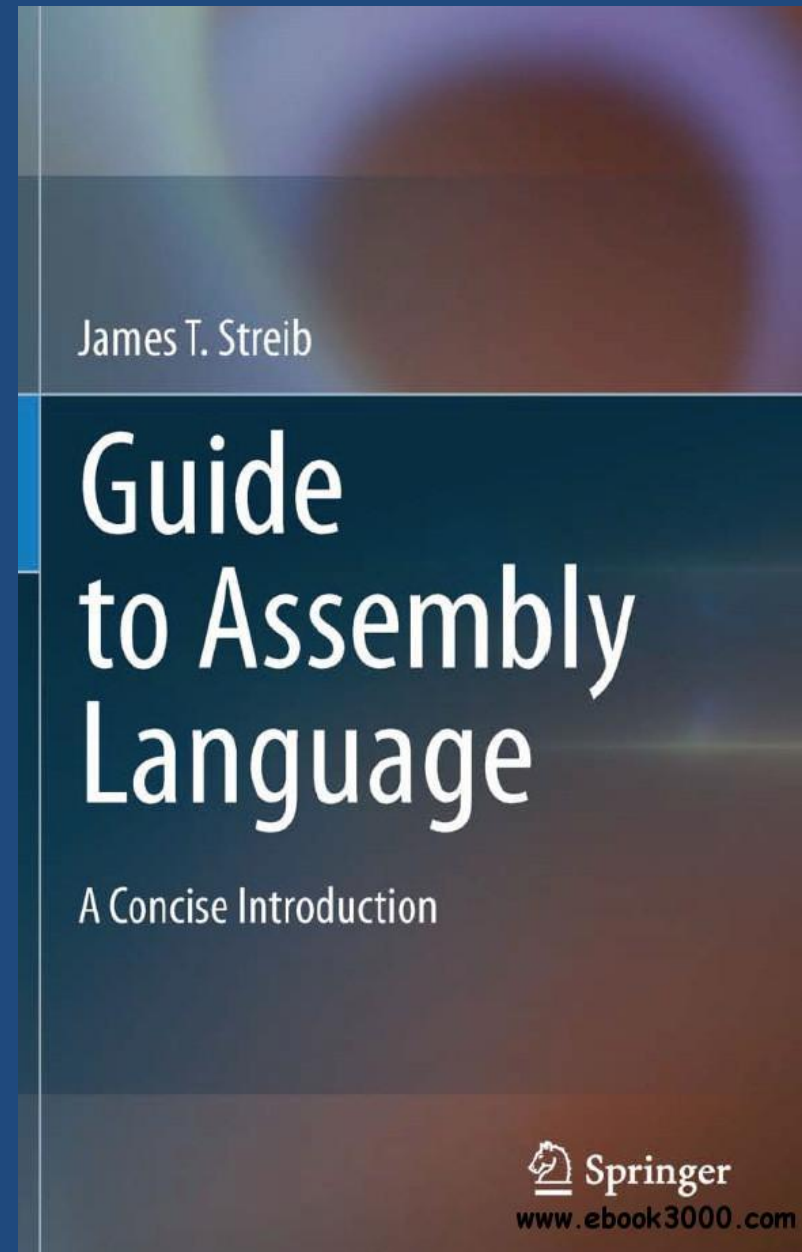
Course Objective

- Introduction to the internal organization of computer systems, including the relationship between
 - Computer hardware
 - It's native instruction set
 - Implementation of high level languages on that machine
- Practical work using the x86 based Microsoft Macro Assembler

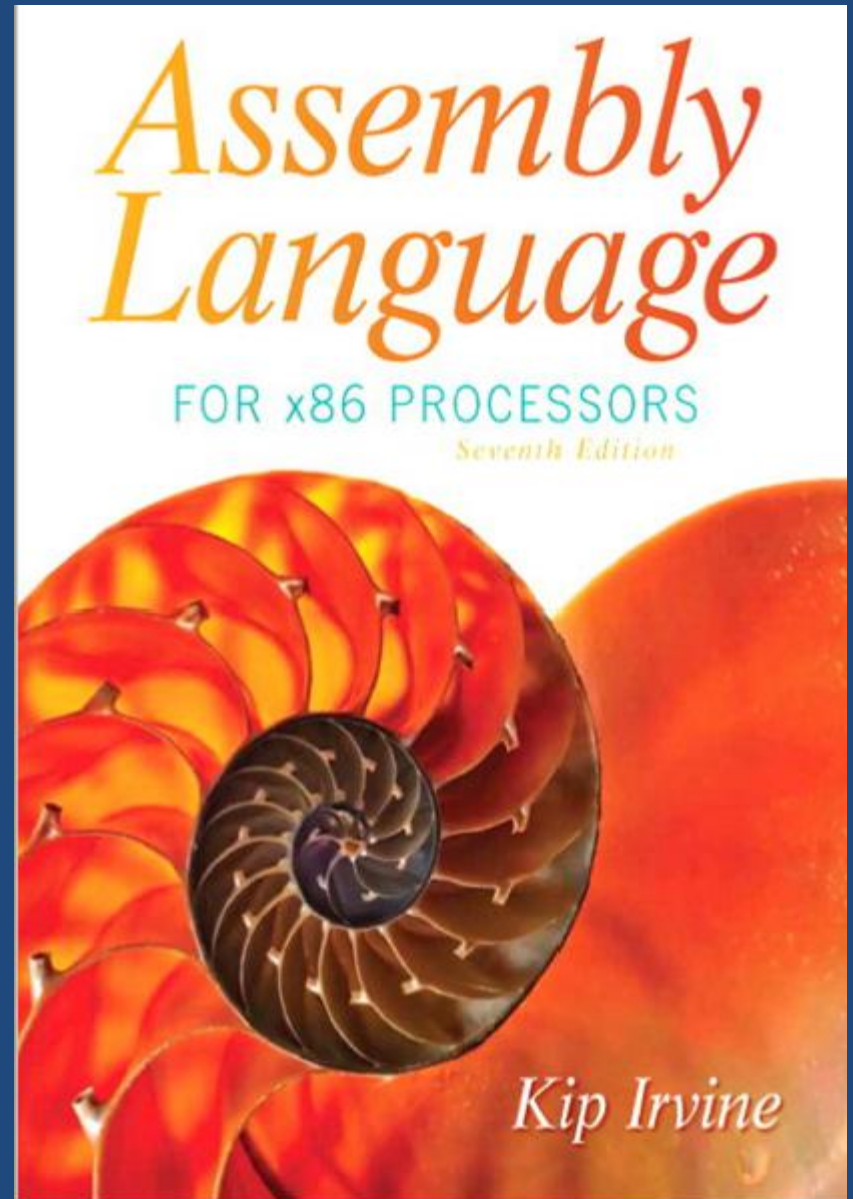
Learning Outcome

- Understanding the process that occurs when high level code is translated into machine code
- A better understanding of how code actually works
- Ability to program using Intel's x86 assembly language

Text Book



Reference Book



Reference Book

Assembly Language Programming Lecture Notes

Delivered by
Belal Hashmi

Compiled by
Junaid Haroon

Course Format

- 2 credit hour course
 - 2 lectures in a week

Lectures	Labs
•Intro to course •A sample first program	Lab: A sample first program
•Binary, hexadecimal, logic and arithmetic revision	Problem set: Conversion practice
•Variable declaration •Registers •Data movement	Lab: Inspecting register and variable values
•Character data •Errors	Lab: A complete program
•Input and output	Lab: A complete program using input, data transfer and output

Lectures	Labs
•Arithmetic instructions •Addition & subtraction •Multiplication & division •Unary operators	Lab: Implementing I/O and arithmetic
•Selection structures •High level directives	Lab: Selection structures
•Iteration structures	Lab: Power function implementation
•Logic instructions (shift, rotate) •Stacks	Lab: Simulating an OCR machine
•Procedures and macros	Lab: Implementing a macro calculator

Lectures	Labs
•Arrays •Declaration •Indexing	Lab: Selection sort
•Strings	Lab: An array of strings
•Interrupts	Lab: Using interrupts
•Selected machine language instructions	Lab: Using machine language instructions
•Recap and revision	Lab: Practice exercises

Division of Marks

Total	100
Assignments	10
Quizes (4)	15
Midterm Test	25
Project	10
Final Exam	40

Google Classroom code: p5vf35j

For text books, slides, assignments, course updates
etc.

Administrative Info

- Be careful with attendance (for both lectures and labs)
- Keep a record of all your assessment till your grade has been finalized
- No Makeup for an announced quiz and assignments
- All activities assigned in class or for home are marked and will contribute to your grade
- Don' make requests for attendance via email
- No tolerance for plagiarism
- Late assignments will not be accepted

Introduction

Importance of Assembly Language

- Practically, the need for assembly language programming has decreased
- However, the need to understand it has not decreased

Importance of Assembly Language

- Some practical programming experience needed to understand the language
- Some high level languages include low level features, but sometimes using assembly language is efficient in terms of speed and memory
- Like programming in machine language, but easier – helps gain insight into computer organization and architecture
- Builds understanding of high level language implementations and compiler construction concepts

- What is the difference between a High Level Language and a Low Level Language?

High Level Languages

- High-level languages are human-friendly. They are, thus, very easy to understand and learn by any programmer. C, FORTRAN, Java
- Closer to natural languages
- Make programs easier to read and write

Low Level Languages

- Closer to machines
- High level to low level language
 - → One to many relationship
 - Language translators (compilers, interpreters) convert one high level instruction into many low level instructions
 - Native language of machines → machine language → low level → 0s and 1s
- Different microprocessors have different machine languages
- Low-level languages are machine-friendly. They are, thus, very difficult to understand and learn by any human.
- Examples include assembly language, machine language

Assembly

- Midway, whereas programming in machine language is extremely tedious, assembly language is still manageable
- Assemblers convert assembly language to machine language

What Are Assemblers and Linkers?

An assembler is a utility program that converts source code programs from assembly language into machine language.

A linker is a utility program that combines individual files created by an assembler into a single executable program.

A related utility, called a debugger, lets you to step through a program while it's running and examine registers and memory.

How Does Assembly Language Relate to Machine Language?

- Machine language is a numeric language specifically understood by a computer's processor (the CPU). All x86 processors understand a common machine language. Assembly language consists of statements written with short mnemonics such as ADD, MOV, SUB, and CALL.
- Assembly language has a one-to-one relationship with machine language.
- Each assembly language instruction corresponds to a single machine-language instruction.

How Do C++ and Java Relate to Assembly Language?

- High-level languages such as Python, C++, and Java have a one-to-many relationship with assembly language and machine language.
- A single statement in C++, for example, expands into multiple assembly language or machine instructions.
- Most people cannot read raw machine code, so in this book, we examine its closest relative, assembly language.
- Understand this through an example

Example

- For example, the following C++ code carries out two arithmetic operations and assigns the result to a variable. Assume X and Y are integers:
- `int Y;`
- `int X = (Y + 4) * 3;`

How to write this code in assembly language?

Example

- Following is the equivalent translation to assembly language. The translation requires multiple statements because each assembly language statement corresponds to a single machine instruction:
- `mov eax,Y ;` move Y to the EAX register
- `add eax,4 ;` add 4 to the EAX register
- `mov ebx,3 ;` move 3 to the EBX register
- `imul ebx ;` multiply EAX by EBX
- `mov X,eax ;` move EAX to X

End of Lecture